

nce for what steps to complete before, during, and after you return from leave. For questions specific to your leave or pay, please contact the Absence Management team in HelpLab. This checklist is currently in its first iteration. Please continue to check back for updates.

State-Specific Leaves

Each US state varies when it comes to types of leave employers are required to allow employees to take. Employees of GitLab Inc. are eligible for our paid time off policy and our Sick Time policy; which will run concurrently with any State or FMLA leave entitlement. Please refer to the Sick Time Procedures - USA or Paid Time Off policy for more information.

The Family Medical Leave Act (FMLA)

The Family Medical Leave Act FMLA provides eligible U.S team members up to 12-weeks, or up to 26- workweeks (in one 12 month period to care for an injured service member), unpaid time away from work for their own serious health condition, to care for a family member with a serious health condition, or to bond with a newborn or newly adopted child. FMLA eligible leave time can be taken in continuous segments or intermittently as certified by a physician. If FMLA is used to bond with a child, or newly adopted child, all eligible time must be used prior to the child's first birthday or date of adoption. For more information please see Employee Rights Under the Family Medical Leave Act.

Team members must have 12 months of continuous service and 1250 hours worked in the year immediately before the start of their leave to be eligible for FMLA leave. Working hours are calculated as actual hours worked and does not include PTO or vacation time. GitLab calculates FMLA eligibility as a "rolling" 12-month period measured backward from the date of any FMLA leave usage. (Each time a team member takes FMLA leave, the remaining leave is the balance of the 12 weeks not used during the 12 months immediately before the FMLA leave is to start.) All FMLA entitled leave will run concurrently with any State specific leave and disability or paid family leave programs (if applicable), GitLab's sick leave policy (including COVID leave), and/or Short-Term disability benefits. GitLab's Flexible PTO policy can not be used to delay, extend, or in lieu of FMLA.

How to Apply

Any medical related absence (for yourself or a family member) requiring you to be away from work for at least 3 but no more than 5 days (either continuously or non-continuous but related to the same condition) must be reported either in Workday or to the Absence Management Team in HelpLab. You will be contacted within 24-48 business hours (whenever possible) and provided with information regarding your request. If approved, please adjust your Slack status noting you are OOO.

Payroll Processing During FMLA

The Absence Management team will review the team members eligibility for Short-Term Disability benefits, State pay benefits, and/or GitLab's Sick Time policy, and will communicate how the team member can expect to receive their pay while on leave.

California Family Rights Act (CFRA)

GitLab will grant time off to eligible team members in accordance with the requirements of the California Family Rights Act ("CFRA") and the federal Family and Medical Leave Act (Fed-FMLA). Where both the CFRA and Fed-FMLA apply, the leave provided by each will count against the team member's entitlement under both laws and must be taken concurrently. A team member who is eligible for leave under only one of these laws will receive benefits in accordance with that law only. In any case, team members will be eligible for the most generous benefits available under applicable law.

The following policy addresses team member rights under the CFRA only. Team members should refer to GitLab's Team Member Handbook for additional detail regarding the Fed-FMLA. All questions concerning this policy should be directed to the Absence Management team in HelpLab.

Team Member Eligibility

To be eligible for CFRA leave, team members who reside in California must have been employed by the Company for a total of at least 12 months (52 weeks) at any time prior to the commencement of the leave and have worked at least 1,250 hours over the previous 12 months as of the start of the leave.

Qualifying Reasons For Leave

Eligible team members may request leave under the CFRA for one or more of the following reasons:

- For the birth of a team member's child or the placement of a child with the team member for foster care or adoption, so long as the leave is completed within 12 months of the birth or placement of the child;
- To care for the team member's spouse or registered domestic partner, child (regardless of age or dependency status), parent, grandparent, grandchild, or sibling, with a serious health condition;
- For the team member's own serious health condition, except for disability from pregnancy, childbirth or a related medical condition; or
- For a qualifying exigency related to the covered active duty or call to covered active duty of an team member's spouse, domestic partner, child, or parent in the Armed Forces of the United States;
 - For purposes of this policy, a "parent" includes a biological, foster or adoptive parent, a stepparent, parent-in-law, a legal guardian or other person who stood in loco parentis to the employee when the employee was a child.
 - "Serious health condition" means an illness, injury (including, but not limited to, on-the-job injuries), impairment or physical or mental condition that involves either:
 - Inpatient care (including, but not limited to, substance abuse treatment) in a hospital, hospice or residential medical care facility, including any period of incapacity (that is, inability to work, attend school or perform other regular daily activities) or any subsequent treatment in connection with this inpatient care; or
 - Continuing treatment (including, but not limited to, substance abuse treatment) or continuing supervision by a health care provider that includes one or more of the following:
 - A period of incapacity (that is, inability to work, attend school or perform other regular daily activities due to a serious health condition, its treatment or the recovery that it requires) of more than three consecutive calendar days, and any subsequent treatment or period of incapacity relating to the same condition, that also involves treatment two or more times

via an in-person visit to a health care provider, or at least one visit to a health care provider that results in a regimen of continuing treatment under the supervision of the health care provider;

- Any period of incapacity or treatment for incapacity due to a chronic serious health condition that requires periodic visits to a health care provider, continues over an extended period of time and may cause episodic incapacity;

- A period of incapacity that is permanent or long-term due to a condition for which treatment may not be effective, such as Alzheimer's, a severe stroke and the terminal stages of a disease; or

- Any period of absence to receive multiple treatments (including any period of recovery) by a health care provider either for (a) restorative surgery after an accident or other injury; or (b) a condition that would likely result in a period of incapacity of more than three consecutive calendar days in the absence of medical intervention or treatment.

Length of Leave

Team members are entitled to a maximum of 12 workweeks of CFRA leave in a 12-month period. The applicable "12-month period" used by the Company is a rolling 12-month period measured backward from the date an employee uses CFRA leave. Under this method the 12-month period is measured backward from the day the employee uses any CFRA leave.

CFRA leave is not available when a team member is disabled by pregnancy, childbirth or a related condition. However, team members disabled by pregnancy, childbirth or a related medical condition may be entitled to pregnancy disability leave under California law and the Federal FMLA. Federal FMLA leave will generally run concurrently with pregnancy disability leave. CFRA leave is in addition to and will not run concurrently with leave taken in accordance with California's pregnancy disability leave law.

When CFRA leave is for the birth or placement of a child and both parents work for the Company, they will each be allowed up to 12 weeks of CFRA leave within 12 months of the child's birth or placement.

When the reason for CFRA leave is the team member's serious health condition, which also constitutes a "disability" under California's Fair Employment and Housing Act ("FEHA"), and the team member cannot return to work at the conclusion of the CFRA leave, the Company will engage in an interactive process to determine whether an extension of leave would be a reasonable accommodation under the FEHA.

Intermittent or Reduced Schedule Leave

Under some circumstances, team members may take CFRA leave intermittently, which means taking leave in blocks of time or reducing the team member's normal weekly or daily work schedule. A team member may take leave intermittently or on a reduced schedule whenever it is medically necessary to care for the team member's child, parent, spouse, registered domestic partner or registered domestic partner's child with a serious health condition or because the team member has a serious health condition. The medical necessity of the leave must be determined by the health care provider of the person with the serious health condition.

Intermittent or reduced schedule leave may also be taken for absences where the team member or their family member is incapacitated or unable to perform the essential functions of the job because of a chronic serious health condition, even if the person does not receive treatment by

a health care provider.

Leave due to military exigencies may also be taken on an intermittent or reduced leave schedule basis.

Leave taken intermittently may be taken in increments of no less than one hour. Team members who take leave intermittently or on a reduced work schedule basis for planned medical treatment must make a reasonable effort to schedule the leave so as not to unduly disrupt the Company's operations. Please contact the Absence Management team in HelpLab prior to scheduling medical treatment. If CFRA Leave is taken intermittently or on a reduced schedule basis due to planned medical treatment, the Company may require team members to transfer temporarily to an available alternative position with an equivalent pay rate and benefits, including a part-time position, to better accommodate recurring periods of leave.

If a team member using intermittent leave or working a reduced schedule finds it physically impossible to start or stop work mid-way through a shift in order to take CFRA leave and is therefore forced to be absent for the entire shift, the entire period will be counted against the team member's CFRA entitlement. However, if there are other aspects of work that the team member is able to perform that are not physically impossible, then the team member will be permitted to return to work, thereby reducing the amount of time to be charged to the team member's CFRA entitlement.

CFRA leave for Bonding Leave does not have to be taken in one continuous period of time, but the minimum duration is two weeks. However, the Company will grant a request for CFRA leave lasting less than two weeks' twice during the 12-week period. Additional requests for Bonding Leave lasting less than two weeks may be directed to the Absence Management team in HelpLab and will be considered on a case-by-case basis depending on the needs of the Company. If the request is granted, the Company may require the team member to transfer temporarily to an available alternative position. Bonding Leave must be concluded within one year of the birth or placement of the child.

Requesting Leave

Team members who wish to take planned family or medical leave must notify the Absence Management team in HelpLab with reasonable promptness when they become aware of the need for leave and should identify the planned dates of the leave. The Company may require team members to provide written notice of the need for leave, except where written notice is not possible because of the need for immediate health care consultation or treatment.

When the need for the leave is foreseeable (such as for the expected birth or placement of a child) team members must, if possible, provide at least 30 days' advance notice. For events that are unforeseeable, team members should notify the Company (at least verbally) as soon as they learn of the need for leave. Employees should provide notice by submitting a ticket to the Absence Management team in HelpLab.

Team members who need CFRA leave that is foreseeable due to a planned medical treatment should make reasonable efforts to schedule leave to avoid disruption to Company operations. In addition to other notice provisions, team members requesting leave for CFRA qualifying reasons must respond to any questions designed to determine whether an absence is potentially qualifying for leave under this policy. Failure to respond to permissible inquiries regarding

the leave request may result in denial of CFRA leave protections

Certification of Health Care Provider

When the leave relates to medical issues (i.e., the serious health condition of a team member or family member), team members will be required to provide a medical certification within 15 calendar days of the Company's request, unless it is not practicable to do so. Certification forms are available from the Absence Management Team by contacting the Absence Management team in HelpLab. Team members on CFRA leave for their own or a family member's serious health condition may be required to provide a recertification when the original certification expires, if additional leave is requested.

At the Company's expense, the Company may also require a second medical opinion regarding a team member's own serious health condition. Team members are expected to cooperate with the Company in obtaining additional medical opinions that the Company may require.

Qualifying Exigency Leave Requirements

Team members are required to provide:

- As much advance notice as is reasonable and practicable under the circumstances;
- A copy of the covered servicemember's active duty orders when the team member requests leave and/or documentation (such as Rest and Recuperation leave orders) issued by the military setting forth the dates of the servicemember's leave; and
- A completed Certification of Qualifying Exigency form within 15 calendar days, unless unusual circumstances exist to justify providing the form at a later date.

Certification forms are available from the Absence Management team by submitting a ticket in HelpLab.

Failure to Provide Notice or Certification and to Return From Leave

Absent unusual circumstances, failure to comply with these notice and certification requirements may result in a delay or denial of the leave. If a team member fails to return to work at the leave's expiration and has not obtained an extension of the leave, the Company may presume that the team member does not plan to return to work and has voluntarily terminated their employment.

Benefits

The Company will continue making contributions for a team member's group health benefits during a leave on the same terms as if the team member had continued to work. This means that, if a team member wants benefits coverage to continue during CFRA leave, the team member must continue to make any premium payments they were required to make for themselves or their dependents prior to the leave. Team members will generally be provided with group health benefits for a 12-workweek period. In some instances, the Company may recover premiums it paid on a team member's behalf to maintain health coverage if the team member fails to return to work following CFRA leave for reasons other than the continuation, recurrence, or onset of a serious health condition or circumstances beyond the team member's control.

A team member's length of service will remain intact, but benefits such as vacation and sick leave may not accrue while on an unpaid CFRA leave.

No loss of benefits accrued prior to the leave will occur as a result of leave under the CFRA, but team members are not entitled to any benefit or position that they would not have been entitled to if they did not take the leave.

Compensation During Leave

Leave taken under this policy is generally unpaid, although depending upon the circumstances, team members may be eligible to receive benefits through state-sponsored programs or the Company's sponsored wage-replacement benefit programs. Also, team members may choose to use PTO and sick leave, to the extent permitted by law and the Company's policy. The Company may require team members to use PTO to cover some or all of the leave, only if the CFRA leave is otherwise unpaid. The CFRA leave is not unpaid if the team member is receiving state disability insurance, short or long term disability payments pursuant to an employer provided plan, or is receiving Paid Family Leave through the state. The use of paid benefits will not extend the length of CFRA leave.

Job Reinstatement

Under most circumstances, team members will be reinstated to the same position they held at the time of the leave or to an equivalent position with equivalent pay, benefits and other terms and conditions of employment. If a team member becomes unqualified during CFRA leave as a result of not attending a necessary course, or renewing a license, the team member will be given a reasonable opportunity to fulfill those conditions upon returning to work. Further, the Company may grant a team member's request to work a different shift, in a different or better position, or in a different location, that is better suited to the team member's personal needs upon returning from CFRA leave. The Company will also consider a reasonable accommodation under the FEHA if the team member is returning from CFRA leave for their own serious health condition. However, team members have no greater right to reinstatement than if they had been continuously employed rather than taken leave. For example, if a team member would have been laid off or the team member's position would have been eliminated even if they had not gone on leave, then the team member will not be entitled to reinstatement. However, if a team member has been replaced or the team member's position was restructured to accommodate the team member's absence, the team member is entitled to reinstatement. The Company will not limit or deny reinstatement from CFRA leave on the basis that an employee is considered a "key employee" under the FMLA.

Prior to being allowed to return to work, a team member wishing to return from leave for their own serious health condition must submit an acceptable release from a health care provider that certifies the team member is able to resume work. For a team member on intermittent or reduced schedule CFRA leave, such a release may be required up to once every 30 days if reasonable safety concerns exist regarding the team member's ability to perform their duties, based on the serious health condition for which the team member took the intermittent or reduced schedule leave.

Confidentiality

Documents relating to medical certifications, recertifications or medical histories of team

members or team members' family members will be maintained separately and treated as confidential medical records, except that in some legally recognized circumstances, the records (or information in them) may be disclosed to supervisors and managers, first aid and safety personnel or government officials

Fraudulent Use of CFRA Leave Prohibited

A team member who fraudulently obtains CFRA Leave from the Company is not protected by the CFRA's job restoration or maintenance of health benefits provisions. In addition, the Company will take all available appropriate disciplinary action against a team member due to such fraud.

Nondiscrimination

The Company takes its CFRA leave obligations very seriously and will not interfere with, restrain or deny the exercise of any rights provided by the CFRA. We will not terminate or discriminate against any individual for exercising their right to family and medical leave under the CFRA or for giving information or testimony regarding their own or another person's leave in an inquiry or proceeding related to rights under the CFRA. If a team member believes that their CFRA rights have been violated in any way, they should immediately report the matter to Team Member Relations.

Team members should contact the Absence Management team in HelpLab as to any CFRA questions they may have.

New York Paid Family Leave (NYPFL)

Paid Family Leave Overview

New York Paid Family Leave provides job-protected, paid time off so you can:

- Bond with a newly born, adopted or fostered child,
- Care for a close relative with a serious health condition, or
- Assist with family situations when a family member is deployed abroad on active military service.

Paid Family Leave may also be available for use in situations when you or your minor dependent child are under an order of quarantine or isolation due to COVID-19. See PaidFamilyLeave.ny.gov/COVID19 for full details.

Benefits

Eligible employees may take up to 12 weeks of leave. This leave can be taken either all at once or intermittently, but must be taken in full-day increments. You may take the maximum time-off benefit in any given 52-week period.

- Paid Family Leave can be taken by employees who are eligible for time off under the provisions of FMLA. PFL will run concurrently with designated FMLA leave and GitLab's 16 weeks Parental Leave, when the reason for leave qualifies under PFL, FMLA, and GitLab's Parental Leave policy. Eligible employees must apply for both PFL and FMLA.
- Generally, you may not receive short-term disability and Paid Family Leave benefits at the same time. You may not take more than 26 combined weeks of short-term disability and Paid

Family Leave in a 52-week period.

- If you are unable to work and qualify for workers' compensation benefits, you may not use Paid Family Leave benefits at the same time as you are receiving workers' compensation benefits. If you are working and receiving reduced earnings, you may be eligible for Paid Family Leave.

Pay

Paid Family Leave benefits provide 67% of your average weekly wage, capped at the same percentage of the New York State Average Weekly Wage. Visit [PaidFamilyLeave.ny.gov](https://www.PaidFamilyLeave.ny.gov) for information on this year's maximum weekly benefit, as well as a calculator to estimate your own benefits.

- EXAMPLE: An employee who earns \$1,000 a week would receive a benefit of \$670 a week.

Eligibility

All eligible employees are entitled to participate in Paid Family Leave. Employees are eligible regardless of citizenship and/or immigration status.

- Full-time employees: Employees who work a regular schedule of 20 or more hours per week are eligible after 26 consecutive weeks of employment.
- Part-time employees: Employees who work a regular schedule of less than 20 hours per week are eligible after working 175 days, which do not need to be consecutive.

Funding

Paid Family Leave is funded through employee payroll contributions that are set each year to match the cost of coverage. The rate of employee contributions is reviewed annually, and is subject to change by the New York State Department of Financial Services. Visit [PaidFamilyLeave.ny.gov](https://www.PaidFamilyLeave.ny.gov) for information on this year's employee contribution rate, as well as a calculator to estimate your own payroll contributions.

Qualifying Events

New Child: You can take Paid Family Leave during the first 12 months following the birth, adoption, or fostering of a child. Expectant mothers cannot take Paid Family Leave for their own pregnancy. Paid Family Leave for the birth of a child begins after the birth. It is not available for prenatal conditions.

Serious Illness: You can take Paid Family leave to care for a family member with a serious health condition. These relatives can live outside of New York State and even outside the country. You cannot take Paid Family Leave for your own health condition. A serious health condition is an illness, injury, impairment, or physical or mental condition that involves:

- Inpatient care in a hospital, hospice, or residential health care facility, or
- Continuing treatment or continuing supervision by a health care provider

A family member includes: Spouse, Domestic Partner, child and stepchild, Parent and

stepparent, Parent-in-law, Grandparent, Grandchild, and effective 2023 Sibling (biological, adopted, half, and step).

Military Active Service Deployment: You can take Paid Family Leave to assist with family situations arising when your spouse, domestic partner, child, or parent is deployed abroad on active military service or has been notified of an impending military deployment abroad. You cannot use Paid Family Leave for your own qualifying military event.

COVID-19 Quarantine: Paid Family Leave may also be available for use in situations when you or your minor dependent child are under an order of quarantine or isolation due to COVID-19. See PaidFamilyLeave.ny.gov/COVID19 for full details.

Your Rights and Protections

- You have job protection, ensuring you can return to the same job (or a comparable one) when you return from Paid Family Leave.
- You can keep your health insurance while on leave. If you contribute to the cost of your health insurance, you must continue to pay your portion of the premium cost while on leave.
- Your employer is prohibited from discriminating or retaliating against you for requesting or taking Paid Family Leave.

Taking Paid Family Leave

1. Notify your employer. When you want to take Paid Family Leave, you must notify your employer at least 30 days before your leave will start if it's foreseeable. Otherwise, notify your employer as soon as possible.
1. Obtain required forms. Contact your employer, employer's insurance carrier, or visit PaidFamilyLeave.ny.gov to obtain the required forms.
1. Complete and attach: The Request For Paid Family Leave (Form PFL-1) has sections that need to be completed by you and by your employer. Fill out your section, make a copy and give the form to your employer to fill out Part B. Your employer is required to return Form PFL-1 to you within three business days. If there is a delay, you do not have to wait to proceed. Send the Form PFL-1 that you have filled out, along with the rest of your request package, directly to your employer's insurance carrier.
1. Obtain and attach supporting documentation. The specific documentation or additional forms required for each type of leave are described on the request for Paid Family Leave and at PaidFamilyLeave.ny.gov/Apply.
1. Submit your request forms and supporting documentation. You must submit your completed request package to your employer's insurance carrier within 30 days after the start of your leave to avoid losing benefits. In most cases, the insurance carrier must pay or deny benefits within 18 calendar days of receiving your completed request or your first day of leave, whichever is later.

Paid Family Leave Process and Contacts

At GitLab, requests for Paid Family Leave forms and forms submissions should be directed to the Absence Management Team in HelpLab. Our Paid Family Leave carrier is NYLife.

Disputes

If your Paid Family Leave claim is denied, the insurance carrier (NYLife) will provide you with information about how to request arbitration. A neutral arbitrator will decide claim-related disputes.

Discrimination Complaints

Employees are protected from discrimination and retaliation for requesting or taking Paid Family Leave. If your employer terminates your employment, reduces your pay and/or benefits, or disciplines you in any way as a result of you requesting or taking Paid Family Leave, send your employer's designated Paid Family Leave contact a formal request for job reinstatement using the Formal Request For Reinstatement Regarding Paid Family Leave (Form PFL-DC-119), which can be found in the forms section of PaidFamilyLeave.ny.gov. File the completed form with your employer and send a copy to:

> Paid Family Leave, P. O. Box 9030, Endicott, NY 13761-9030

If your employer fails to comply with the request for reinstatement within 30 days, you may file a Paid Family Leave discrimination complaint with the Workers' Compensation Board using Paid Family Leave Discrimination Complaint (Form PFL-DC-120), which is also available on the Paid Family Leave website. Once your complaint is received, the Board will assemble your case and schedule a preliminary hearing in front of a Workers' Compensation Law Judge.

Other Leave Types By State

Leave Type	State or City/Region
-----	-----
Bereavement Leave	IL, OR, WA (Tacoma)
Blood/Bone Marrow/Organ Donation Leave	AR, CA, CT, HI, IL, LA, ME, MD, MN, NE, NJ, NY, OR, SC, WI
Court Attendance/Witness Leave	CA, CT, DC, FL, GA, HI, IL, IN, IA, KY, MD, MA, MN, MO, NV, ND, OH, OR, PA, RI, TX, UT, VT, VI, WI, WY
Crime Victims' Leave	AL, AK, AZ, AR, CA, CO, CT, DE, MD, MA, MI, MN, MS, MO, MT, NH, NY, OH, OR, PA, RI, SC, VT, VA, WY
Domestic/Sexual Violence Victims' Leave	AZ, AR, CA, CO, CT, DC, FL, HI, IL, IA, KS, ME, MD, MA, MI, MN, NV, NJ, NM, NY, NC, OR, PA, RI, TX, VT, VA, WA
Drinking and Driving Class Leave	IA
Drug/alcohol Rehabilitation Leave	CA
Election Officials' Leave	AL, CA, DE, IL, KY, MN, NE, NC, OH, VA, WI
Emancipation Day Leave	DC
Emergency Evacuation Leave	TX
Legislative/Political Leave	CT, IA, ME, MN, MT, NV, OK, OR, SD, TX, VT, WV
Literacy Leave	CA
Mobile Support Unit Leave	IN
Paid Family/Disability Leave	CA, CO, CT, DC, DE, HI, MA, MD, NH, NJ, NY, OR, Puerto Rico, RI, VA, WA
Pregnancy Disability Leave	CA, CT, HI, IA, LA, MN, MT, NH, OR, WA
Public Health Emergency (Quarantine/Isolation) Leave	AZ, CA (San Diego), CO, DE, IL (Cook Co), MA, MD (Montgomery Co), MI, MN, NJ, NY (Westchester Co & NYC), OR, PA (Pittsburgh), RI, SC, VT, WA

School Activities/Visitation Leave	CA, CO, DC, IL, LA, MA, MN, NV, NJ, NC, OR, RI, TN, VT
State Family Medical Leave	AK, CA, CT, DC, FL, GA, HI, IL, IN, IA, KS, KY, LA, ME, MD, MA, MN, MO, MT, NE, NV, NH, NJ, NM, NY, NC, OH, OR, PA, Puerto Rico, RI, TN, VT, VA, WA, WI
Student Leave	DC
Volunteer Emergency Responder Leave	AL, CA, CO, CT, DE, IL, IN, IA, KS, KY, LA, ME, MD, MA, MO, NE, NV, NH, NJ, NM, NY, NC, ND, OH, OR, PA, RI, SC, TN, UT, WA, WV, WI
Voting Leave	AL, AK, AZ, AR, CA, CO, DC, GA, IL, IA, KS, KY, MD, MA, MN, MO, NE, NV, NM, NY, ND, OH, OK, SD, TN, TX, UT, VT, WV, WI, WY

U.S Military Leave

GitLab, Inc. recognizes the obligation of those team members serving in any branch of the military or other uniformed services of the United States. Employment status within GitLab is protected by the Uniformed Services Employment and Reemployment Rights Act of 1994 ("USERRA").

Leave and Re-Employment

Team members who serve on active or reserve duty will be granted a leave of absence up to the maximum time required by law. GitLab is committed to preserving the job rights of team members absent on military leave in accordance with law.

Compensation

Military leave runs concurrently with GitLab PTO. Team members on unpaid military leave will be paid at 100% of their salary for the first 25 days of leave. Exempt team members will not incur any reduction in pay for partial week absences for leave under this policy.

Health Care Continuation

During a military leave of less than 31 days, a team member is entitled to continued group health plan coverage under the same conditions as if the employee had continued to work. For military leaves of more than 30 days, an employee may elect to continue their health coverage in accordance with USERRA and COBRA. For additional information on health care continuation contact the Absence Management team in HelpLab.

Notification of Supervisor/Manager

Team members needing 3 or more consecutive business days away from work due to Military Service are expected to provide GitLab with as much advance notice as possible. In some instances providing advance notice may not be possible, so we ask team members to notify us as soon as they are able.

Team Members should report their Military Service in Workday by selecting the Military Leave option under the Leaves dropdown.

- Note: For absences resulting in more than 3 consecutive business days away from work, team members will need to provide a copy of your orders as soon as you receive them. Please submit a ticket to the Absence Management team in HelpLab.

No Retaliation

Team members who request military leave will not be retaliated against or penalized in any manner. Any team member who believes they have been retaliated against in violation of this policy should notify the Absence Management team in HelpLab immediately.

SECTION: product/_index.md

Welcome to the Product Team Handbook

- The Product Team is led by David DeSanto.
- The Product Handbook focuses on how we do product at GitLab.
- Please see Product Direction to learn about what the Product Team plans to build.

Product Team Functions

- Product Management
- User Experience
- Product Technical Program Management

Product Team Mission

We create products and experiences that customers love and value.

- Consistency wins as you scale. Our organizational goal is to create a Product Manager (PM) system that maximizes the chances of success across new products, new features, and new team members.
- We are shipping an experience, and not just a product. Don't forget about the links between the product and the website, pricing & packaging, documentation, sales, support, etc.
- It's about our customers and doing a job for them, not the product itself. Think externally about customer problems, not internally about the technology.
- It's about love AND value. Will customers value what we are building? We need to make sure what we build helps build and extract customer value.

Contributing to the Product Handbook

The product handbook is widely referenced by product managers and cross-functional team members. Therefore, we want to follow a consistent change management process so all affected collaborators are aligned and informed when there are changes (specially new requirements, processes, meetings, etc.) that affect their workflow.

It is helpful to indicate which type of change you are suggesting:

- Small improvement (typos, clarifications, etc.)
- Adding a new section
- Modifying existing section
- Documenting a new process
- Adding a new page or directory

- Other

For a small improvement, please feel free to merge the MR yourself. Otherwise, please tag the appropriate code owner for review and approval. The guidance below should be followed for driving awareness:

Informing team members about the changes

It is your responsibility to communicate with relevant team members about your merge request. Here are some best practices to follow:

- For significant changes affecting the whole Product or Engineering team, consult your manager or product leader for the best communication strategy.
- For changes specific to product management, tag @gl-product-pm in the merge request before merging.
- Indicate in the comment whether you are requesting contributions and feedback or simply providing information.
- For changes relevant to other teams, tag department leads such as the VP of UX, VP of Development, or the Director of Quality Engineering in the merge request before merging.
- Share and cross-post the merge request link with a brief description in relevant channels such as Slack product, product-leadership, eng-managers, and ux-managers.
- For more guidance on communication, refer to the GitLab Communication tips and best practices.

Product Management

If you'd like to collaborate with product management see How to Engage guide.

Product Principles

The Product Principles section is where you can learn about our strategy and philosophy regarding product development here at GitLab.

Product Processes

For a detailed view on how we do Product Development, read up on our established Product Processes.

Product sections, stages, groups, and categories

To learn how the GitLab product and our Teams are organized review our Product Categorization section.

About the GitLab Product

Learn about GitLab as a product, including what does it mean to be a single application, our subscription tiers and pricing model, and the basics of permissions in the platform.

Product Manager Responsibilities

Understand the roles and responsibilities of product managers.

Being a Product Manager at GitLab

Want to know more on what being a Product Manager at GitLab is like? Checkout our Product Management guide for helpful information like our Career Development Framework and learning/development resources.

Product Performance Indicators

Learn how we measure success in Product via our Product KPIs, which are tracked in our Product project. For best practices and guidance on how to add instrumentation for features please review our Analytics Instrumentation workflow.

Product OKRs

Understand the OKR Process for the GitLab Product Team and review current and past OKRs.

Our Product Leadership Team

Learn about our Product Leadership Team and learn about them via their personal README's.

Communicating with the Product Division

Below are team emails and handles that can be used for different departments and sub-departments in the Product Division. These groups are used for internal communication and the @mention can only be used by project members. Please remember that tagging @mention on issues will generate in-product to-do items and email notifications to all team members in that project, so use it only when you need to communicate with the entire team. For communication specifically for product managers, please leverage How to Engage.

- @gl-product-leadership tags all group managers, directors and VPs in the Product Division
- @gl-product-plt tags all direct reports to the VP Product in the Product Division
- @gl-product-pm tags to all members of Product Management, Product Monetization and Program Managers in the Product Division
- @gl-product tags to all members in the Product Division: Product Management, Product Monetization, Program Management, Product Design, User Research, and Technical Writing
- product@gitlab.com emails all members of Product Management, Product Monetization and Program Management in the Product Division
- ux-department@gitlab.com emails all members of UX (UX Research, Technical Writing and Product Design) in the Product Division
- @gitlab-com/gitlab-ux tags all members of UX (UX Research, Technical Writing and Product Design) in the Product Division
- @gitlab-com/gitlab-ux/managers tags all people managers within UX (UX Research, Technical Writing, and Product Design)
- @gitlab-com/gitlab-ux/designers tags all Product Designers and Design Managers in UX
- @gl-docsteam for all Technical Writers in UX
- 'technical-program-management' a place to reach the entire TPgM team across organizations

When you are tagging @mention:

- Clearly state why you are tagging the entire product team and what action you need product team members to take.
- Write a short summary in the same comment so team members can quickly understand the necessary context.
- Review the issue title and description to ensure it has relevant details other product team members need BEFORE submitting the comment. The issue title will be the subject of email notifications and in-product to-do items.
- If asking team members to review a change, please directly link to the specific page on the review app and any relevant issues or MRs.

SECTION: product/product-principles.md

{{% include "includes/product-handbook-links.md" %}}

Our Product Principles

These are core principles that we believe will deliver world-class products through customer-centric innovation. Our goal is to build a practice that fosters these principles with the customers' voice at the core. Everything we do is for our customers, and we are only successful when they succeed in delivering secure software faster to their customers and internal users.

1. We are customer zero, therefore we use our own product: Everything you put into the product should be a feature you or our Engineering team would use as part of our daily work. If the answer is no, then challenge your "why", as there might be a better solution that will have a bigger impact for our customers.

1. We are not our only customer: Meet with customers as much as possible as everything we do is for them. It is tempting to assume that we understand our customers through our own usage and dogfooding, but that only gets you so far and we can be wrong. Validate assumptions through strategic user research, customer interviews, and feedback sessions.

1. We are design-led: It does not matter what customer pain points you work with Engineering to resolve if what is provided to customers is hard (or almost impossible) to use. User experience is our top priority as we have a very technical product, however it should be easy enough for someone new to DevSecOps to get started quickly. This includes everything from onboarding to shipping secure software leveraging GitLab.

1. We value quality over velocity: Delivering incomplete features and capabilities to ensure your team achieves a defined velocity is not acceptable. Everything we ship to customers must be user validated, bug free, not introduce security vulnerabilities, be able to achieve GitLab.com scale, include documentation, and be available on all customer deployment choices simultaneously. Accruing additional technical debt in favor of velocity is also not acceptable as this leads to future quality issues around availability, scalability, reliability, and security.

1. We value data over intuition and anecdotes: Everything we build must have success metrics we can track to ensure we are providing value to our customers. We measure outcomes, not launches, and this is only possible with experimentation and proper instrumentation. All features must be instrumented so success metrics can be tracked and adjustments to plans can be made through product usage.

1. We fail fast and iterate with intention: Define a hypothesis on how to address a customer use case or pain point and validate (or invalidate) it quickly through problem validation. Take

the outcome of the problem validation cycle and build an iteration strategy to deliver it with a focus on usability and quality. Revalidate your hypothesis through solution validation with each iteration so you can adjust plans as needed. Problem and solution validation ensures the customer voice is key in making decisions.

1. We believe in product-led growth over unguided experiences: Our product should be GitLab's best Sales team member, and its own biggest champion. Enable feature discovery moments to let customers know there is more value based on their usage, actions taken, or configuration choices. The more customers adopt our product's capabilities, the more return on investment they experience, which builds more internal GitLab champions.

1. We like to win and we only win as a team: We win when our customers are able to successfully ship secure software faster with GitLab. This requires us to hold ourselves to the highest levels of execution and collaboration within Product across our Product Management, UX Research, Product Design, and Technical Writing teams. While teamwork within Product is necessary, it is also not sufficient. We require that same standard of execution and collaboration with our cross-functional team members across R&D and GTM. Together, as a global GitLab team, we can drive Results for Customers.

How we follow our Principles

Enabling Collaboration

From development teams to marketing organizations, everyone needs to collaborate on digital content. Content should be open to suggestions by a wide number of potential contributors. Open contribution can be achieved by using a mergeable file format and distributed version control. The mission of GitLab is to allow everyone to collaborate on all digital content so people can cooperate effectively and achieve better results, faster.

Bringing Ideas to Reality

Ideas flow through many stages before they are realized. An idea originates in a chat discussion, an issue is created, it is planned in a sprint, coded in an IDE, committed to version control, tested by CI, code reviewed, deployed, monitored, and documented. Stitching together all these stages of the DevOps lifecycle can be done in many different ways. You can have a marketplace of proprietary apps from different suppliers or use a suite of products developed in isolation.

As a single-application for the entire DevOps lifecycle, GitLab strives to enable you to bring ideas to production rapidly. We will do so AND avoid demonstrating our capabilities with toy app demos and simple trivial examples. That's because we understand that building a prototype is easy, but building a production line is hard.

The Minimal Valuable Change (MVC)

Minimal Valuable Change (MVC) is the GitLab path to delivering the smallest measurable improvement for our users, customers, and the wider community.

Our approach requires four pillars:

- Relentless customer focus and commitment to understanding their workflows, using research and validation
- Measurable outcomes that use established metrics for success in tracking adoption, usage, or other business outcomes.
- Product functionality that adheres to GA criteria listed in the levels of support
- Future vision to expand the MVC beyond the initial release

When considering how to scope a feature for a release, remember that it is not ok to ship an "incomplete" feature to customers (see the definition of done). Consider the use of Pajamas components for UI in your MVCs. When introducing a new component or pattern not found within Pajamas, it is the responsibility of that team to follow our component lifecycle guidelines to determine whether it should be added and, if so, contribute the addition/update back to Pajamas.

MVC means reducing the scope so we can ship quickly. It doesn't mean shipping something that hurts the usability of GitLab. First impressions are important. A feature that does not offer enough value or hinders the user experience may have a negative effect that discourages users from trying that feature again in the future. If there are obvious gaps in your MVC or you can anticipate follow-up requests, consider whether your feature is complete enough to be released to users. If you are unsure whether your feature is complete enough to be an MVC (or if you know your feature is not complete enough to be an MVC and you want to gather additional feedback), you can use approaches such as dogfooding, beta programs, feature flags, and/or user research to help build confidence in your decision. In terms of talking about your feature, it's ok to add a release post item that announces your incomplete feature (making clear that it is an early iteration, and points to the direction for the feature) and follow up in a later release post with a new item when you've completed more of the functionality. As long as you call it cookie dough, not a cookie, it manages user expectations.

Examples:

- Ship a feature through the API and not the UI - See this release post as a great example of this approach was used to build a GraphQL endpoint for deleting attachment from a project.
- Exposing a minimal set of functionality - See this release post where a basic read-only page displaying queued jobs was added and more capabilities were added in subsequent releases.

There are scenarios when an MVC approach is not advised. These include:

- When changing core parts of the experience - An example of a core experience is comments. When building this out for work items, we waited to release the new feature to end users until we reached parity with comments in Issues and MRs.

Iteration

An MVC approach is a byproduct of our spirit of iteration. That means we break problems down as small as possible, and focus on reduced cycle time. Thinking iteratively is not always intuitive, and breaking certain topics or projects down can be challenging. Here's a helpful video from our CEO with guidance on how to think more iteratively.

Here is a great video that illustrates how to build MVCs using Iteration. It shows Lego climbing obstacles. The first design fails. The second one can climb a book and so on. It also

illustrates how modularity and good interfaces help with iteration as things get complex.

Iteration speed and product excellence

To ensure that our efforts consistently deliver value to our users, each iteration must adhere to the following guidelines:

1. Define Expected Impact: Clearly articulate the expected measurable positive impact of the iteration on users, ensuring alignment with our overall product direction and guided by a vision that has been informed by user research.

1. Establish Metrics for Success: Identify specific metrics that will be used to evaluate the success of the iteration before declaring the feature GA or an iteration shippable. These should be concrete, measurable indicators that directly relate to the intended outcomes of the iteration.

- These metrics should define a measurable quality bar as part of the initial scoping as that allows the cross-functional team to understand what the success/quality criteria are before beginning to build, while also allowing these metrics to be measured during the entire development and post-release lifecycle. As part of defining the quality bar a test plan should be defined and agreed that it can measure against these measures for success. Quality targets include:

- No S1 or blocking S2 defects/bug that prevents the customer from using the feature
- Not impacting or severely degrading the stability of the GitLab instance

Success of an initiative is not measured by the deployment of a change or the completion of an iteration. True success is determined by whether the iteration achieved its predefined objectives, as evidenced by tangible business and product metrics.

Example:

- Effort: Reduce latency in Service A.
- Iteration A: Implement regional upgrades in key locations.
- Success Metrics: Evaluate the iteration's success by measuring increased usage and user satisfaction with Service A. Relevant metrics might include service usage rates, user adoption levels, repeat usage statistics, and revenue increases following the upgrade.

We celebrate achievements when we can clearly see that an iteration or launch has resulted in tangible value for our users.

Subtractive thinking

Humans tend to favor solutions that add features than solutions that remove them, even when removing features is more efficient - great PMs recognize this bias and utilize subtractive thinking to create great user experiences. Customers will tell us when we're missing something they need, but they're unlikely to explicitly tell us when we're overwhelming them with unwanted features. However, we do have evidence that this challenge is already a consideration for us, as reflected consistently in our System Usability Scale verbatims. Here is an episode on the Hidden Brain podcast that explores this bias further.

SaaS First

Our customers choose SaaS because it reduces their operating costs, helps them adopt the latest capabilities without performing upgrades, and provides them peace of mind of high availability. This principle implies the following:

- Design features so that they can be released without downtime.
- Release features in SaaS before self-managed or alongside it.

This principle does not mean SaaS only. For more information on parity between SaaS and self-managed, see our parity principle.

Feedback issues

An MVC approach allows for maximum feedback while iterating. To help gather that feedback, Product Managers are encouraged to create feedback issues (example) to consolidate suggestions and experiences from users. Consider mentioning the feedback issue in any release post items and related implementation issue(s) for awareness.

- Feedback issues allow GitLab team members and the wider GitLab community to offer thoughts and suggestions for future iteration.
- Feedback issues are particularly encouraged for major new customer-facing features.
- These issues can be closed at the end of the milestone following the milestone when they were introduced.

Celebrate and Learn from Failures

It's important that team members know they are encouraged by E-group to continue moving fast as GitLab grows and gets bigger as a company. This involves moving quickly even in the face of risk and complexity. In alignment with our transparency value, we'd like to celebrate examples of failures or mistakes that were made while moving fast, where we ultimately learned from it and moved on.

The following failures provided by the product team are celebrated as opportunities to gain insight, share learnings and move on with additional knowledge:

1. We initially believed we should create a lift and shift translator for Jenkins, but learned from users and technical experts that it wasn't technically feasible and it would be better to invest in more detailed documentation and guidance.
1. With Auto DevOps, we discovered the desire for composability and the need to iterate on your DevOps platform makes a one-pipeline fits all capability miss the mark for tackling that user pain.
1. We integrated with some monitoring and observability tools as MVCs (Jaeger) that were just too MVC to give us data points on their success.
1. We relied upon "MVC" not including on by default and usable by default - resulting in lots of MVCs that we thought would garner us new information, but did not.
1. We spent time delivering a marketplace offering, and had very little traction. We learned marketplace offerings alone are not a vehicle for adoption, and we need a sales alignment with the marketplace vendor to achieve success.
1. We had duplication of effort between Quality and Distribution. We didn't recognize / resolve this for too long. We have learned to better integrate Quality into the planning process.
1. Data telemetry in general has been one of our failures. We didn't invest in telemetry

seriously or early enough, and we attempted to accelerate via a 3rd party, but didn't do a good enough job talking to our community on the best ways to roll that out.

1. We failed to capture significant users with APM. Internal Deck for Reference

1. Historically, designers had been instructed to not spend time on problem validation, but we had a PM who realized in order for her group to be successful, she needed her design counterpart to be deeply involved in validation work.

1. We learned that GitLab's original Serverless strategy was dependent on immature technology and not necessarily aligned with the market winner Lambda. This resulted in GitLab stopping investment in Serverless.

1. We recognized a big gap in security concerns with GitLab managed clusters, which was prohibiting this feature from being adopted by customers. Once we learned from this 'failure' we introduced the Kubernetes agent instead.

If you have a failure that can serve as a learning opportunity, please make an MR to this page

Avoid "Not Invented Here" Syndrome

Just because something is not invented here doesn't mean it doesn't have a perfect home within our solution. GitLab is an Open Core product and is part of the broader ecosystem of Open Source tools in the market. Every day there are new innovative open source tools out there that solve real-world customer problems; we should not be afraid of embedding these tools into our own products in order to solve those same problems for our customers too. Leveraging existing technology allows us to get to market much more quickly, to contribute to Open Source (and help strengthen Open Source as a whole), and allows us to focus our own people on making GitLab itself better. Building professional relationships with these tool creators also is a positive for GitLab since they may have important user perspectives around your categories.

We have achieved many successes following this approach:

- Code Quality in CI/CD pipelines by embedding CodeClimate
- Feature Flags by using the Unleash client library
- Mobile publishing by writing about how to utilize FastLane with GitLab

There are also many more examples throughout the company where this has been successful. As a product manager you should be monitoring the world of Open Source as it relates to your area to see where new innovative tools are being developed, and not be afraid of integrating those. One thing to keep in mind, integrating could be anything from a blog post describing how the tool works together with GitLab all the way up to bundling it inside of our own installation, and this can evolve iteratively.

Convention over Configuration

We understand that a natural inclination when using application development tools is to create an array of buttons to press and knobs to turn. We believe, however, that adding options to an application does not necessarily improve the user's experience of that application. The best way to serve our users is to create an application that reduces complexity while still offering the features they need.

Inspiration

We admire other "convention over configuration" tools like Ruby on Rails (the doctrine of which perfectly describes the value of integrated systems), Ember, and Heroku and we strive to offer the same advantages for continuous delivery of software.

Furthermore, Ruby on Rails has significantly and positively influenced the Ruby community, uplifting the tool and making it more powerful and useful than ever before. We want GitLab to be to Kubernetes what Rails is to Ruby.

You should prefer well-considered choices based on current best practices. Avoid unnecessary configuration. Avoid configuration to support fragile workflows.

Configuration Principles

When considering adding new configuration, we follow the following principles:

- Ensure a great experience by default - GitLab should work perfectly right out of the box for most users. While it should be resisted, sometimes configuration is inevitable or preferable. Your configuration must not make that experience worse and should always get out of the way of the user.
 - GitLab.com values should be the default - the settings used on GitLab.com should be the defaults for self-managed. Not only does this provide a consistent experience for users, but we get the highest fidelity feedback via GitLab.com. If we find that the GitLab.com setting is wrong, it is usually wrong for self-managed too. If you believe you have a strong case for using custom (non-default) settings for GitLab.com, please document the justification in alignment with your Product Section Lead. Any custom (non-default) settings for GitLab.com need to be tracked [here](#).
- Encourage favorable behaviors by limiting configuration - Convention also implies that we're encouraging our customers to do things in a certain way. A very concrete example of this is the ability to disable pipelines. We believe that our integrated solution will give a superior user experience and we're motivated to encourage this behavior. For this reason, adding a configuration to allow disabling this permanently (be that in a template or instance-wide), is something that should be avoided.
- Design for users not intermediaries - GitLab should avoid falling into the Blackboard trap of building a product that administrators of GitLab love because it is configurable, but developers and other users of GitLab hate because it is overly complex and confusing.
- Working by Default - From the perspective of someone using GitLab, the feature does not exist until it works by default. This means, with few exceptions, features should simply work on GitLab.com and self-managed installations of GitLab without setup, toggling feature flags, modifying GitLab Omnibus (`gitlab.rb`) or Charts configurations, or installing extra components. This is more difficult than "enabled by default," which implies that features may be available by default but then require additional effort to set up. Working by default is worth the extra thoughtfulness and effort, because it enables an extremely important outcome: It allows customers to easily adopt our entire platform and experience the benefits of a single app for the entire DevOps lifecycle. In order to be properly working by default a feature needs two things:
 - Enabled-by-default - it must not require modifying GitLab Omnibus (`gitlab.rb`) or Charts configurations, installing extra components on the host machine, or be behind a feature flag. When features are not enabled-by-default most people will never benefit from it because admin

access to the GitLab application or the host machine will be needed. Feature flags should be consistently ON for both GitLab.com and self-managed users whenever possible.

- Enabled-by-default can be rolled out incrementally. A feature can sometimes be activated within days via a feature flag on GitLab.com. Other times a feature can take months to prove that it has the performance and visibility enterprises need.

- Setup-by-default - it should not require the setup before the feature can be used. We should ensure every feature has sensible defaults, and automatically migrate existing users/groups/projects to have the new features setup by default, while ensuring that the security and infrastructure costs would not be substantially affected. It is important to remember that most people are not likely to take the extra effort to setup a new feature, nor will they be aware that the feature has been added. The vocal proponents of issues who a Product Manager might interact with would likely be inclined to take the extra effort to setup a feature, but most people will not.

- Avoid Limits - Limits should be in place to protect the system but not to "slowly try out" a feature. By limiting the usefulness of a feature right from the start the only thing you are achieving is limiting its adoption and usefulness. If you are going to default to OFF or limited, you must have a good, documented reason for this.

- Avoid configuration completely when possible - Requests for configuration can be a proxy for trying to support a fragile workflow. Rather than enabling bad habits and incurring product debt, effort should be spent helping customers adopt best practices.

- Configuration builds up over time - Every configuration option in GitLab multiplies its complexity, which means the application is harder to use, harder to develop, and less friendly to users.

- Configuration is hard to remove - Removing a configuration after it shipped and in use is much more work than not introducing it in the first place. This is because you change the behavior for customers that selected the less popular option.

- Configuration is an expensive testing mechanism - It's a natural reaction to propose a big change to be configurable, as you worry it'll negatively affect certain users. However, by making a feature configurable, you've now created two problems to maintain going forward. Adding a configuration is a one way door that should be avoided if possible. As a result, consider using feature flags instead of configuration.

Always Allow for Deploying to Production

Sometimes fast deployments are needed to fix a service or application outage that can cost a business money and reputation, we understand time is of the essence in these situations. That's why we believe giving the team control over this is important in crucial moments of the development lifecycle. Controls that prevent changes from reaching Production are okay as safeguards, but they should be able to be quickly removed or disabled if necessary. When controls are changed in this way, logs or records should be created to support post-mortem analysis and allow for understanding why the control needed to be removed or disabled.

<figure class="videocontainer"><iframe
src="https://www.youtube.com/embed/03ODv1cEO6E"></iframe></figure>

Design for self-managed for feature parity between deployments

We want to provide the same capabilities to end users regardless of the method they choose to use GitLab (GitLab SaaS, Dedicated or Self-managed). All GitLab SaaS environments leverage the same installation method available to self-managed users, with a different licensing structure. By designing and implementing features for self-managed, we achieve maximum parity between the

various i